

Notebook on
Software Engineering

Prepared by
Shahriar Arefin

ID: 22101040

Section: (A)

Faculty
Jawad Hossain

Semester: Fall 2024

Contents

1	Security and Maintenance	1
1.1	Software Security	1
1.2	Web-based attacks	1
1.3	Security Technologies	2
1.3.1	Digital Signature	2
1.3.2	Algorithms in Digital Signature	2
	Software Engineering Quick Recap	3

Chapter 1

Security and Maintenance

1.1 Software Security

Software security is the application of **techniques that assess, mitigate, and protect software systems from vulnerabilities.**

- **Confidentiality:** It protects against disclosure of information to unintended recipients. *Solution:* Encryption, access control, authentication.
- **Integrity:** It allows transferring accurate and correct desired information from senders to intended receivers. *Solution:* Hashing, digital signatures, and checksums.
- **Availability:** It ensures readiness of the information on requirement. *Solution:* Redundancy, fault tolerance, backup, regular maintenance, and protection against DDoS attacks.



1.2 Web-based attacks

Mnemonic: In Dhaka, Smart People Buy Delicious Dumpling Using Fresh Money

Injection attacks	: Attacker injects malicious code into a web application and fetch required information.
DNS Spoofing	: Attacker changes the DNS address of a website to redirect the user to a fake website.
Session Hijacking	: By stealing the cookies, an attacker can have access to all of the user data.
Phishing	: Steals sensitive information like user credentials and credit card number by pretending to be trustworthy sources.
Brute Force	: A hacking method that uses trial-and-error to crack passwords, credentials, or encryption keys.
Denial of Service (DOS)	: Attacker overloads a target server or network with traffic from multiple sources, making it unavailable to legitimate users.
Volume-based attacks	: Its goal is to saturate the bandwidth of the attacked site, and is measured in bit per second.
Protocol attacks	: It consumes actual server resources, and is measured in a packet.
Application layer attacks	: Its goal is to crash the web server and is measured in request per second.
Dictionary attack	: Attacker uses a list of commonly used passwords to guess original one.
URL Interpretation	: Attacker changes certain parts of a URL, leading to unauthorized access or data exposure.
File Inclusion Attack	: Attacker uses the include feature to access sensitive files or run malicious code on the web server.
MITM	: Attacker to intercepts connection between client and server and acts as a bridge between them.

1.3 Security Technologies

1.3.1 Digital Signature

A digital signature is a mathematical technique which validates the authenticity and integrity of a message, software or digital documents. It allows us to verify the author name, date and time of signatures, and authenticate the message contents. The digital signature offers far more inherent security and intended to solve the problem of tampering and impersonation (Intentionally copy another person's characteristics) in digital communications.

1.3.2 Algorithms in Digital Signature

- **Key generation algorithm:** The key generation algorithm selects private key randomly from a set of possible private keys. This algorithm provides the private key and its corresponding public key.
- **Signing algorithm:** A signing algorithm produces a signature for the document.
- **Signature verifying algorithm:** A signature verifying algorithm either accepts or rejects the document's authenticity.

Software Design

Software design is a process to **transform user requirements** into some suitable form, which helps the programmer in **software coding and implementation**.

Objectives: Correctness, Completeness, Efficiency, Flexibility, Consistency, and Maintainability

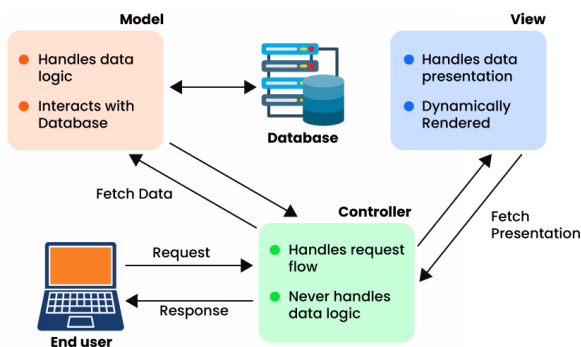
Design Principles: Problem Partitioning, Abstraction, Modularity, and Top Down & Bottom up Strategy

Design Principles vs Patterns

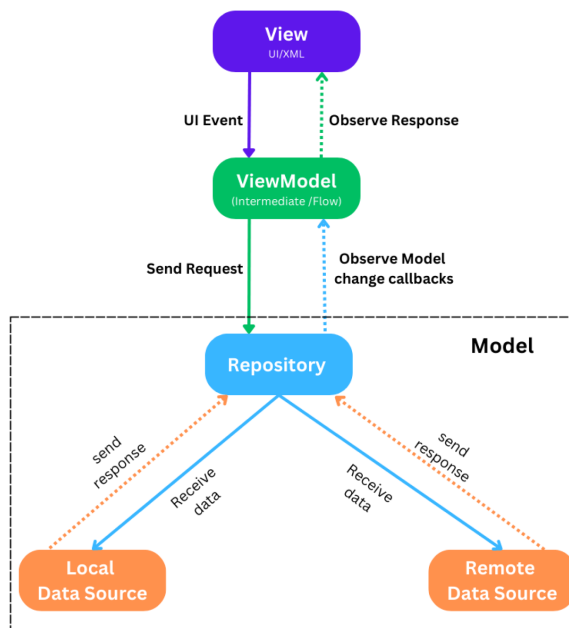
Design Principles are **general guidelines** that can guide your class structure and relationships. • Design patterns are **proven solutions** that solve commonly reoccurring problems.

MVC

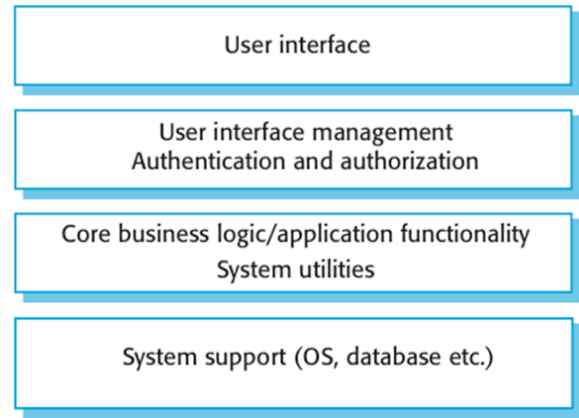
Model - Handles data and business logic • **View** - Handles UI and presentation • **Controller** - Handles input and updates model/view



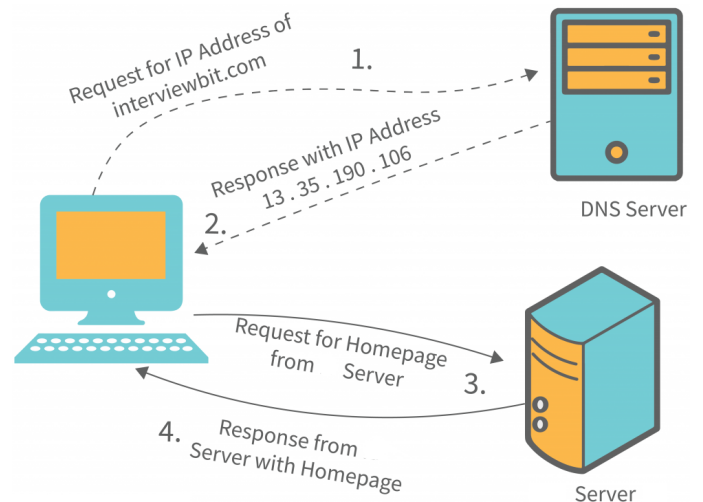
MVVM



Layered Architecture



Client Server Architecture



Layered Architecture | 3-Tier Architecture

- Organizes software into layers, each with specific responsibilities.
- Communicates via adjacent layers (top-down or bottom-up)
- Scaling individual layers is complex.
- Use case: MVC, 3-tier systems

Client-Server Architecture | 2-Tier Architecture

- Organizes system into clients (requesters) and servers (providers).
- Communicates over a network (e.g., HTTP, sockets).
- Servers/clients can scale independently.
- Use case: Web Apps, Cloud Services, Email system

Coupling & Cohesion

Coupling | Inter-Module Binding

- Shows relationships **between modules**
- Relative independence between modules should be less
- Aim for **low coupling** (minimal dependencies)
- Modules are linked to other modules

Cohesion | Intra-Module Binding

- Shows relationship **within a module**
- Measures module's functional strength
- Aim for **high cohesion** (single function focus)
- Module has little interaction with others
- Focuses on a single responsibility

Dependency Injection

Dependency Injection is the process of **providing an object (dependency) that a class needs**, rather than letting the class create it itself. Its used to achieve Inversion of Control (IoC) between classes and their dependencies.

Involves four roles: **Service**, **Client**, **Injector**, and **Interface**.

Ways of DI: **Constructor**, **Setter**, and **Interface**.

Verification, Validation & Testing

Verification & Validation

Verification | Static Testing

Are we building the product right? • Verify product meets design specs. • Quality assurance team does verification.

Validation | Dynamic Testing

Are we building the right product? • Ensure product satisfies user needs. • Testing team does validation.

Cleanroom process | F I S S S

Formal specification, **Incremental** development, **Structured** programming, **Static** verification, and **Statistical** testing.

Inspection process

Remember: Please Only Intelligent People In Red Fashion

Planning, Overview, Individual, preparation, Inspection, meeting, Rework, and Follow-up

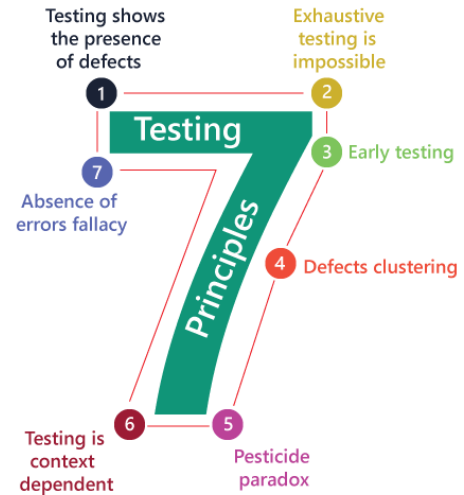
Software testing

Verifying software correctness by assessing all attributes: reliability, scalability, portability, reusability, and usability.

STLC: Requirements, Planning, Designing, Setup, Execution, Reporting, and Closure.

Benefits of Software Testing

Cost-Effective, Security, Product Quality, and Customer Satisfaction.



Black box testing | Functional / Behavior Testing

Testing software functionality **without looking at internal code**, focusing only on inputs and outputs.

Techniques: Decision Table, Boundary Value, State Transition, All-pair Testing, Cause-Effect, Error Guessing, and Use Case.

Generic Steps: Requirement Analysis, Test Scenario Design, Test Case Development, Test Execution, Result Comparison, Defect Handling

White box testing | Structural / Logic Testing

Testing software by looking inside the code, checking logic, conditions, loops, and internal structures.

Techniques: Statement Coverage, Path Test, Flow Graph Notation, and Loop Testing.

Gray box testing | Hybrid Testing

Testing with partial knowledge of internal code and user perspective — a mix of black box and white box testing.

Techniques: Matrix, Pattern, and Regression Testing.

Cost Estimation

We know that,

$$E = a(KLOC)^b \times EAF \quad D = c(E)^d \quad SS = \frac{E}{D} \quad P = \frac{KLOC}{E}$$

Therefore, $EAF \propto E$ and $E \propto D$

Experience Level $\uparrow$ $EAF \downarrow$ $E \downarrow$ $D \downarrow$

Complexity Level $\uparrow$ $EAF \uparrow$ $E \uparrow$ $D \uparrow$

Tools $\uparrow$ $EAF \downarrow$ $E \downarrow$ $D \downarrow$